

Dify 實作手冊：九個範例與長照 3.0 專案

Dify Hands-on — 9 Examples & the LTC 3.0 Case

M8 講義教你「為什麼」，這份手冊教你「怎麼做」。

每個節點填什麼、每個參數設多少、程式碼長什麼樣——全部給到可照做的層級。

本手冊大綱

01 ★ 開始之前：Dify 是什麼 · 定位、情境與核心概念

02 ★ Dify vs n8n 怎麼選 · 適用情境對照與能力分工

03 本手冊與 M8 的分工 · 觀念 vs 操作

04 環境建置四件事 · Docker · Dify · API · 嵌入模型

05 Workflow vs Agent · 五個維度的對照

06 Ex1-Ex5 五種應用型態 · 從文字生成器到 Chatflow

07 Ex6-Ex9 模型與參數 · 模型無關性、Temperature、JSON

08 模型選型 · 不同節點用不同模型

09 工具與外部整合 · Slack · Email · MCP · Function Calling

10 知識庫分段策略 · 為什麼要用分隔符

11 三種檢索策略 · 混合檢索為什麼是黃金標準

12 ★ 長照 3.0 四層架構 · Input · Data · Logic · Output

13 ★ 三個知識庫的參數 · 逐項給到數值

14 ★ 七個節點完整編排 · 含 System Prompt 與程式碼

15 外籍看護排他條款 · 一個真實的法規邏輯闡

16 測試逐字稿與預期反應 · 含陷阱題

17 ★ 一個真實的失敗 · 手冊自己記錄下來的

本手冊與 M8 講義的分工

同一個長照 3.0 案例，兩份講義切入的層級不同：一份講「為什麼這樣設計」，一份講「每個欄位填什麼」

M8 講義 (92 頁)

「為什麼」——建立判斷力

- 四種應用型態的取舍與選型
- RAG 為什麼會失效、五種失效模式
- Agent 思考迴圈的原理
- Prompt 對什麼有約束力、對什麼沒有
- 期末專題規格與評分

★ 適合：課堂講述、建立判斷力

本手冊 (依 Dify tutorial.pdf)

「怎麼做」——照著操作

- Docker 與 API 服務逐步安裝
- 九個 Example 的實際操作步驟
- 知識庫參數要填什麼數值
- 完整可貼上的 System Prompt 與程式碼
- 長照 3.0 七節點完整 SOP

★ 適合：實機操作、照著做

★ 建議用法：W13-W15 課堂用 M8 建立觀念，課後用本手冊實機操作。

兩份的長照 3.0 案例是同一個，但本手冊給到「每個欄位填什麼」的層級。

★ 本手冊最有價值的部分在最後：它記錄了一次「真實的失敗」——

AI 文字說 CMS 8 級、JSON 卻寫 6 級，還自行編造了價格。

那一頁比任何成功範例都有教學價值，因為它示範了「拿到輸出之後要檢查什麼」。

M8 講義 (92 頁)

為什麼：四種型態的取舍、RAG 五種失效模式、Agent 原理、Prompt 的約束力邊界。

★ 本手冊

怎麼做：Docker 安裝、九個 Example 操作步驟、知識庫參數填什麼數值、完整可貼上的 Prompt 與程式碼。

建議用法

W13-W15 課堂用 M8 建立觀念，課後用本手冊實機操作。

★ 最有價值的部分在最後

它記錄了一次真實的失敗——AI 文字說 CMS 8 級、JSON 卻寫 6 級，還自行編造了價格。

九個 Example 與一個綜合專案

九個範例分成三組，最後全部組合成一個真的能用的系統

A 五種應用型態

- Ex1 **Slogan Generator**
文字生成器：最單純的起點
- Ex2 **AI Chatbot (healthcare)**
對話 + 記憶 + 知識庫
- Ex3 **Agent**
工具 (Tools) 與自主性
- Ex4 **Marketing Plan (Workflow)**
固定路徑的自動化
- Ex5 **Movie Researcher (Chatflow)**
對話 + 工具 + 記憶

B 模型與參數優化

- Ex6 **Model Agnostic**
不同節點換不同模型
- Ex7 **Temperature / Max Tokens**
兩個最重要的參數
- Ex8 **Response Format (JSON)**
把輸出變成可程式化資料
- Ex9 **Model Component**
依任務挑模型的邏輯

★ B 組都是「同一個流程換設定」——
先把 A 組跑通，再回頭調參數

C 工具整合與知識庫

- **Tools 組件**
把 LLM 變成行動執行者
- **Slack / Email 整合**
結果送到日常通訊軟體
- **知識庫與 RAG**
分段、索引、檢索策略
- **MCP / Function Calling**
工具呼叫的標準化

★ D 綜合專案：台灣長照 3.0 智慧評估系統

三個知識庫 + 七個節點 + 完整 System Prompt 與 Python 程式碼 + 測試逐字稿
本手冊約三分之一的篇幅在這個案例上——也是期末專題最好的參考範本。

A 五種應用型態

Ex1 文字生成器 → Ex2 Chatbot → Ex3 Agent →
Ex4 Workflow → Ex5 Chatflow。

B 模型與參數優化

Ex6 模型無關性、Ex7 Temperature/Max Tokens、
Ex8 JSON 回應格式、Ex9 模型組件選用。

C 工具整合與知識庫

Tools 組件、Slack/Email、RAG、MCP 與 Function Calling。

★ D 綜合專案

台灣長照 3.0 智慧評估系統——
本手冊約三分之一的篇幅，也是期末專題最好的參考範本。

PART 00

開始之前： Dify 是什麼

Before You Start — What Dify Is

直接跳進操作步驟很容易迷路。

這一章先把「它是什麼、什麼時候該用它」講清楚，後面照做才有方向感。

Dify 是／不是

Dify 是

- ▶ LLM 應用的組裝平台——你在上面接線，不是訓練模型
- ▶ 視覺化流程編輯器，概念上很接近 AI Studio 的流程圖
- ▶ ★ 內建完整 RAG 知識庫：上傳文件即可做檢索增強，並標註來源
- ▶ 支援多家模型供應商，可隨時切換底層 LLM 而不動流程
- ▶ 應用發布後自動成為一組 REST API，可被其他系統呼叫
- ▶ 有完整 Tracing 日誌，看得到每個節點在想什麼、花了多少 Token

Dify 不是

- ▶ 不是模型訓練工具——它不做預訓練，只做應用層
- ▶ 不會讓爛 Prompt 變好——垃圾進、垃圾出的規則依然適用
- ▶ 不保證正確性——LLM 會幻覺，Dify 只是幫你組裝與把關
- ▶ ★ 不是通用自動化平台——排程、批次、跨系統串接是 n8n 的強項
- ▶ 不是免費的——每次呼叫都在燒 Token，成本要自己控制
- ▶ 不能取代領域知識——知識庫的品質決定答案的品質

一句話定位：Dify 專注在「把 LLM 變成一個可用的應用」；它把 RAG、Prompt 管理、對話狀態這些麻煩事都內建好了，讓你專心處理領域問題。

工具定位圖： Dify 在哪個位置



讀圖方式：越往右＝越難學；越往上＝能做的事越多。Dify 刻意選在左下偏中的位置——放棄一部分彈性，換來「做 LLM 應用時最省事」。這個取舍對本課程的專題正好適合。

Dify 適合處理的六種情境

情境 01

知識庫問答

- 把機構的手冊、規範上傳成知識庫
- 使用者用自然語言問，系統依原文回答
- ★ 可標註來源段落，供人核對

情境 02

衛教與客服 Chatbot

- 回答民眾的常見問題
- 用 System Prompt 約束回答範圍
- 超出範圍時誠實說「查無資料」

情境 03

文件生成與摘要

- 把長篇逐字稿整理成結構化摘要
- 依固定格式產出報告（如 SOAP）
- 批次處理需搭配 n8n

情境 04

結構化判定與試算

- 依規範判定等級或分類
- ★ 用 Code 節點做精算，數字不交給 LLM
- 長照 3.0 專案就是這一類

情境 05

多工具 Agent

- 讓 LLM 自己決定要查什麼、做什麼
- 可串搜尋、計算、外部 API
- 適合任務模糊、需組合多種能力時

情境 06

內部工具的對話介面

- 替既有系統加一層自然語言入口
- 不必重寫系統，用 API 串接即可
- 發布後自帶網頁版與嵌入元件

共同特徵：這六種情境的主角都是「語言理解」與「知識檢索」。如果你的需求主角是排程、批次或跨系統搬資料，那應該找 n8n—— 下一頁詳細比較。

六個一定要先懂的 Dify 概念

CONCEPT 01

應用 App

- Dify 的最小產品單位
- 四種型態：Chatbot / Agent / Workflow / Chatflow

CONCEPT 02

知識庫 Knowledge

- 上傳文件後可被應用檢索
- 切塊、嵌入、索引都在這裡設定
- 一個知識庫可被多個應用共用

CONCEPT 03

節點 Node

- Workflow / Chatflow 的組成單位
- LLM、知識檢索、Code、條件分支等
- 用連線決定執行順序

CONCEPT 04

變數 Variable

- 節點之間傳遞資料的容器
- 開始節點的表單變數是使用者輸入
- 後續節點用它組 Prompt 或做判斷

CONCEPT 05

模型供應商

- 在設定中填入各家 API 金鑰
- 不同節點可指定不同模型
- 切換模型不需重做流程

CONCEPT 06

發布 Publish

- 改完流程要按發布才會生效
- 發布後同時產生網頁版與 REST API
- ★ 忘記發布是最常見的「改了沒反應」

這六個詞會貫穿整份手冊。特別注意最後一個——Dify 的編輯畫面是「草稿」，沒按發布，你在外部呼叫到的仍是舊版本。

Dify vs n8n：這個需求該用哪一個？

實務需求	建議工具	理由
長照個案的智慧評估問答系統	Dify	核心是知識庫檢索＋專業判斷，RAG 是主戲
化療副作用衛教問答機器人	Dify	要掛衛教手冊知識庫並標註來源段落
內部工具的自然語言查詢介面	Dify	單一對話入口，Dify 的 Agent 設定最快
每天自動彙整住院人次並寄報表	n8n	核心是排程＋跨系統資料搬動，沒有語言理解需求
衛教文章自動同步到三個社群平台	n8n	純粹的多服務串接，一個 LLM 都不需要
每晚把新文件自動送進知識庫	n8n	排程＋檔案處理是 n8n 強項；Dify 1.10 起才有簡易排程
★ 一次替 200 位個案跑完長照初評	Dify + n8n	Dify 做單筆專業判斷，n8n 負責批次、重試與寫回
新法規公告後自動比對並通知承辦	Dify + n8n	n8n 監看與通知，Dify 做語意比對與摘要

★ 判斷口訣：問「這件事的主角是『知識與對話』，還是『流程與系統』？」主角是知識與對話 → Dify；主角是流程與系統 → n8n；兩者都是主角 → 兩個一起用。

Dify 與 n8n 的能力分工

比較項目	Dify	n8n
RAG / 知識庫	★ 內建完整，上傳文件即可用	有節點可組，但要自己拼向量流程
Prompt 管理	★ 專屬編輯器，含變數與範例管理	寫在節點參數裡，較陽春
對話狀態	★ 內建會話與記憶管理	需自己接 Memory 子節點
排程與批次	1.10 起有排程觸發；批次與重試仍弱	★ 強項：Schedule、迴圈、分批、重試都內建
串接外部服務	有限，多半靠 HTTP 節點	★ 400 + 現成節點，幾乎什麼都能接
發布形式	網頁版 + 嵌入元件 + REST API	Webhook URL + 內嵌聊天視窗
兩者相接的方式	發布後被 n8n 用 HTTP 呼叫	呼叫 Dify，或提供 Webhook 給 Dify 當工具

★ 實務上最常見的架構：把「專業能力」封裝在 Dify（它已經是一個可被呼叫的 API），把「流程與整合」交給 n8n。兩邊各自可獨立維護、獨立測試——詳細作法見 n8n 補充教材的整合實作章節。

讀這份手冊的建議路徑

第一次接觸 Dify

- ▶ 先讀完這一章建立方向感 → 依 PART 01 把環境裝好 → 從 Ex1 到 Ex5 依序做過一遍
- ▶ ★ 不要跳著做：Ex1-Ex5 是刻意由淺入深，跳過中間會在長照專案卡住

已經有基礎、想直接做專題

- ▶ 略讀 PART 02-05，直接跳到 PART 06 的長照 3.0 四層架構
- ▶ 特別留意節點 4 的 Code 精算引擎與三層防呆——那是整個系統可信度的來源

★ 最有價值的部分在最後

- ▶ PART 07 記錄了一次真實的失敗：AI 文字說 CMS 8 級、JSON 卻寫 6 級，還自行編造價格
- ▶ 看懂那一頁，比看懂任何一個成功範例都有用

PART 01

環境建置

Setup

四件事要準備好。多數卡關發生在這一步。

環境建置：四件事

依序準備：容器環境 → 部署 Dify → 取得 API 金鑰 → 選好嵌入模型



★ 四件事都做完之後，用這三個檢查確認環境真的可以用



★ **金鑰安全：**申請後請自行保管，不要寫進任何會交出去的檔案。

Dify 的 DSL 匯出檔 (.yaml) 不含金鑰，所以繳交作業是安全的——但截圖時要注意有沒有拍到。

① Docker Desktop

Windows 需先啟用 WSL2。驗證指令 docker version，
★ 要看到 Client 與 Server 兩個版本才算成功。

② Dify 部署

本地用 docker compose；不想自架可直接用 dify.ai 雲端版。

③ LLM API 金鑰

Gemini 與 GroqCloud 都有免費額度。
★ 手冊範例多用 Groq。

④ 嵌入模型

RAG 必需。★ 中文知識庫要選支援多語的（如 text-embedding-3-large、bge-m3）。

免費額度的實際限制（以 Groq 的 gpt-oss-120b 為例）

限制項目	額度	實際意義
每分鐘請求 RPM	30 次	課堂練習綽綽有餘；多人同時測可能會排隊
每天請求 RPD	1,000 次	★ 每天 1,000 次對話，多數邏輯推理任務夠用
每分鐘 Token TPM	8,000（中文約 0.5-1.3 萬字）	★ 最容易頂到的一項：長訪視逐字稿 + 知識庫召回可能一次就吃掉大半
每天 Token TPD	200,000（中文約 13-33 萬字）	一天可處理約 25 份完整的訪視評估

★ 實務提醒：如果測試時常常失敗，先看是不是頂到 TPM。解法是降低 Top-K、縮短逐字稿，或改用付費模型。

PART 02

Ex1-Ex5：五種應用型態

Five Application Types

同一個平台，五種做法。先跑過一遍，你就知道差別在哪。

Workflow vs Agent：手冊的五維度對照

特性	AI Workflow	AI Agent
路徑設計	預設的固定路徑（Pre-defined）	★ 自行規劃的動態路徑（Dynamic）
執行邏輯	If-Then 邏輯或固定序列	推理、規劃與自我反思（Reasoning & Loop）
工具使用	在特定步驟被動調用	★ 根據需要主動選擇適用的工具
穩定性	★ 高（輸出可預測）	變數多（但能解決更複雜的問題）
除錯與維護	★ 容易：路徑固定，Tracing 一致	困難：每次執行路徑都可能不同

★ 手冊另外列了 Workflow 勝過 Agent 的理由：可靠性與一致性、成本效率、易於除錯維護、可擴展性。醫療場域這四項都很重要。

Ex1-Ex5：五個範例做什麼、學什麼

範例	應用型態	要學的重點	對應檔案
Ex1 Slogan Generator	文字生成器	最單純的起點： Prompt + 變數 + Temperature	Slogan Generator-1 / -2.yml
Ex2 AI Chatbot (healthcare)	Chatbot	★ 對話記憶、開場白、 {{Tone}} 變數、掛知識庫	AI Chatbot (healthcare).yml
Ex3 Agent	Agent	★ 工具（Tools）與自主性（Autonomy）	Component-Agent-01 / -02.yml
Ex4 Marketing Plan	Workflow	Start → LLM → End 的節點串接	Marketing Plan Generator.yml
Ex5 Movie Researcher	Chatflow	★ 對話 + 工具 + 開啟記憶功能	Movie Researcher (Wiki) / (Tavily).yml

★ 建議照順序跑。Ex1→Ex2 學會「對話與記憶」，Ex3 學會「工具」，Ex4→Ex5 學會「流程編排」——最後全部組合起來就是長照專案。

Ex2 的三個關鍵設定（手冊特別強調）

① 對話記憶：Chatbot 優於文字生成器的關鍵

- ▶ 文字生成器每次呼叫都是獨立的，記不住上一句
- ▶ ★ Chatbot 會把對話歷史帶進 Context，所以能接續對話
- ▶ 驗證方法：問一句、再問「我剛才說什麼」——記得才算成功

② {{Tone}} 變數：幫 AI 換上不同的人格面具

- ▶ 把語氣做成變數，同一個應用可切換「專業／親切／簡潔」
- ▶ ★ 也可以直接寫進 System Prompt，兩種做法各有適用場景
- ▶ 變數的好處：使用者可自己選；寫死的好處：不會被亂改

③ 掛知識庫：從「憑印象回答」變成「有依據回答」

- ▶ Chatbot 介面下方就有 Knowledge 區塊可掛載
- ▶ ★ 掛上之後，回答才有辦法標註來源
- ▶ 這是 Ex2 與後面長照專案之間的橋

PART 03

Ex6-Ex9：模型與參數優化

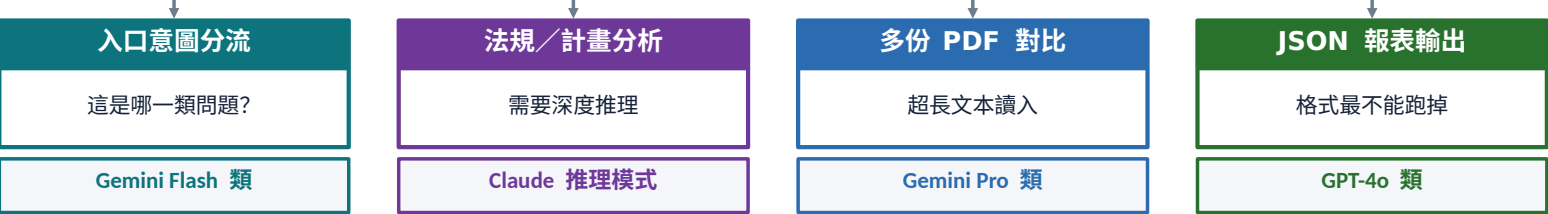
Model & Parameter Tuning

同一個流程，換個模型或改個參數，結果差很多。
這一節把手冊的參數建議整理成可直接套用的表。

模型選型：不同節點用不同模型

同一個流程裡，每個節點各自挑最適合的模型——這是 Ex6 模型無關性的實際用途

同一個 Chatflow —— 每個節點都可以獨立指定模型供應商



流程中的步驟	建議模型	選擇原因
入口意圖判斷	Gemini Flash 類	速度最快，能瞬間判斷需求並分流
專業法規／研究計畫分析	Claude（推理模式）	分析深度與邏輯嚴謹性，減少出錯
多份 PDF 報告對比	Gemini Pro 類	超大長文本，一次讀入所有對比文件
★ 產出標準化 JSON 報表	GPT-4o 類	★ 指令遵循最穩定，格式最不會跑掉
課堂練習（免費額度）	Groq 上的開源模型	★ 每日 1,000 次請求，多數任務夠用

★ 依節點任務挑模型：意圖判斷用快的、法規分析用強的、格式輸出用穩的、練習用免費的。

※ 具體型號變動很快（本表整理自手冊 2026 年版）；真正要記的是「依節點任務挑模型」這個原則。

★ 這是 Ex6 「模型無關性」的實際用途

同一個流程裡，
不同節點可以用不同供應商的模型。

依任務挑

意圖判斷用快的、法規分析用強的、
格式輸出用穩的、練習用免費的。

不是全部用最貴的

入口分流節點用高階模型是浪費——
它只需要判斷「這是哪一類問題」。

★ 真正要記的

具體型號變動很快，但「依節點任務挑模型」這個原則不會變。

Ex7：兩個最重要的參數

參數	作用	調低	調高	醫療場域建議
Temperature	控制輸出的隨機性	可重現、結構穩定	多樣、有創意但會飄	★ 0-0.3。評估與判定類一律低溫
Max Tokens	限制單次輸出長度	可能被硬生生截斷	浪費額度、回應變慢	依輸出格式估算，留 20% 餘裕

★ 手冊的提醒：Max Tokens 設太小時，JSON 會被截斷成不合法的字串，下游的 Code 節點就會解析失敗——這是很常見但不好查的錯誤。

Ex8：把輸出變成可程式化的資料（JSON 模式）

從「非結構化文字」到「下游節點能直接讀」的關鍵一步

JSON 模式的兩個坑

【為什麼要做這件事】

LLM 預設輸出一段自然語言。人看得懂，但程式讀不了。

★ 只要下游有 Code 節點、有條件分支、要接 API —— 就必須先結構化。

【怎麼開】

在 LLM 節點的模型參數中，把 Response Format 設為 JSON。

【★ 手冊特別點出的坑】

開啟 JSON 模式後，提示詞中「必須包含 JSON 這個單詞」，

否則系統會直接報錯：

Messages must contain the word 'json' in some form

→ 解法：在 System Prompt 裡寫「請輸出一個 JSON 物件」即可。

【★ 另一個常見坑：模型會加上 Markdown 標記】

即使開了 JSON 模式，有些模型仍會輸出：

```
```json
{ "cms_level": 6 }
```
```

→ 前後那三個反引號會讓 `json.loads()` 直接失敗。

→ 解法有兩層：

① Prompt 明寫「僅輸出 JSON 字串，不要輸出 Markdown 標記（````）」

② ★ Code 節點裡仍要做清洗（防呆）：

```
clean = s.replace('```json', '').replace('````', '').strip()
```

★ 這兩層都要做。只靠 Prompt 是不夠的 —— 這正是 M8 說的

「Prompt 對格式有約束力，但不是保證」。

PART 04

工具與外部整合

Tools & Integration

把 LLM 從「文本生成器」變成「行動執行者」。

Tools 組件：三種來源

| 來源 | 是什麼 | 範例 | 什麼時候用 |
|---------|----------------------|-----------------------------------|---------------|
| 內建工具 | Dify 已整合好的常用工具 | Google 搜尋、Wikipedia、Tavily、DALL·E | ★ 優先用這些，設定最簡單 |
| 第三方外掛 | Marketplace 上的社群外掛 | Slack、Gmail、Notion、各家 API | 要接特定服務時 |
| ★ 自定義工具 | 自己寫的 API 規格（OpenAPI） | 院內系統、自建資料庫查詢 | ★ 要接機構內部系統時必用 |

★ 手冊示範了兩個外部行動整合：Slack Webhook（把結果發到頻道）與 Email 工具（寄送報告）。兩者都是「AI 從聊天進化為實作者」的具體例子。

Workflow 與 Agent 在「工具參數處理」上的根本差異

Workflow：你來填參數

- ▶ 工具在特定步驟被動調用
- ▶ ★ 參數由你在節點設定中明確指定（或綁定上游變數）
- ▶ 例：搜尋工具的 query 欄，你綁定 `{{#start.keyword#}}`
- ▶ 優點：可預測、可重現、好除錯
- ▶ 限制：你必須事先知道要查什麼

Agent：它自己決定參數

- ▶ 工具由 LLM 根據目標主動選擇
- ▶ ★ 參數也由 LLM 自己生成——它決定要搜什麼關鍵字
- ▶ 例：目標是「研究這部電影」，它自己決定查哪些詞、查幾次
- ▶ 優點：能處理模糊任務
- ▶ ★ 限制：參數對不對你事前不知道，只能事後看 Tracing

★ 這個差異決定了醫療場域的選擇：需要可稽核就用 Workflow，因為「它查了什麼」是可以事先審核的。

MCP 與 Function Calling：手冊的兩個進階主題

Function Calling：從鬆散描述到標準化結構

- ▶ ★ 過去：用自然語言描述工具，模型回傳格式常常不聽話
- ▶ 現在：用結構化定義（name / description / parameters）
- ▶ ★ 好處：穩定性提高、Token 成本更低（不用寫長長的描述）
- ▶ 並可搭配自動重試迴圈：格式不對就重來

MCP：解決「每個 Agent 都要重寫一遍工具」的問題

- ▶ ★ 統一的通訊橋樑：多個 Agent 連到同一個工具伺服器
- ▶ 工具、資料資源、提示詞都可以共享
- ▶ 實務意義：院內做了一個「查詢病歷 API」的工具，所有 Agent 都能用，不用各自實作一遍

★ 對課程專題的意義

- ▶ 這兩個主題屬於【自學】層級，期末專題用不到
- ▶ 但它們解釋了「為什麼 2026 年 Agent 生態突然成熟」
- ▶ M8 的 MCP 那一頁與 M10 的 Vibe Coding 都會再提到

PART 05

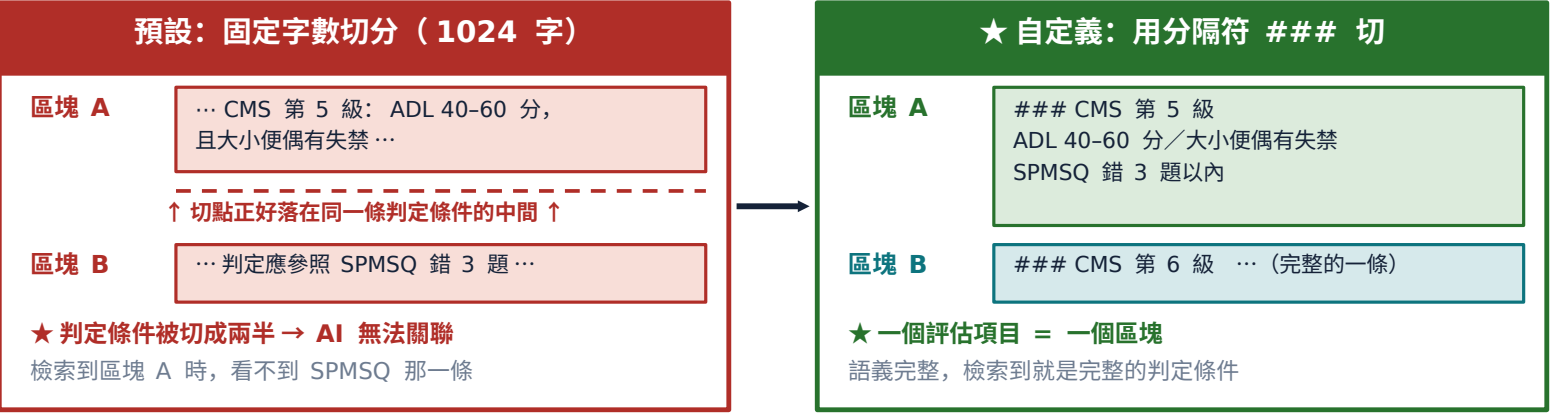
知識庫與 RAG 進階

Knowledge Base & Advanced RAG

手冊在這一節給到的參數細節，是 M8 沒有的。
照著設，你的知識庫品質會差很多。

分段策略：為什麼要用分隔符而不是固定字數

同一份《CMS 評估標準》用兩種切法之後，知識庫裡實際存下來的東西



手冊建議的實際參數（長照 CMS 知識庫）

| | | | | | |
|------|--------------------|--------|------------------------|------|---------------|
| 分段設定 | Custom，分隔符 ### | 分段最大長度 | 1024 字元 | 分段重疊 | 50 字元 |
| 索引方式 | ★ 高品質 High Quality | 嵌入模型 | text-embedding-3-large | 檢索模式 | ★ 混合檢索 Hybrid |

★ 目標：達成「一個檔案 = 一個區塊」或「一個條文 = 一個區塊」。

重疊 50 字元是為了避免語義在斷點被切碎；過大會混入噪音、過小會遺失上下文。

索引方式一定要選「高品質」才會做向量化——經濟模式只做關鍵字比對。

預設固定字數的問題

1024 字處切斷，可能把一條判定標準切成兩半——
AI 檢索到前半段時看不到後半段。

★ 解法：用 ### 當分隔符

達成「一個評估項目 = 一個區塊」，
語義完整，檢索到就是完整的判定條件。

重疊 50 字元

避免語義在斷點被切碎。
過大會混入噪音、過小會遺失上下文。

★ 索引方式選「高品質」

會調用嵌入模型做向量化。
經濟模式只做關鍵字比對，搜「無法走路」找不到「平地走動 0 分」。

檢索策略：三種方式與選用

使用者問：「林陳阿公的 CMS 等級是多少？」——三種找答案的方式

向量檢索

Vector Search

比對語義空間的數學距離

○ 懂近義詞：搜「困難」也能找到「艱辛」

× 對專有名詞不敏感（人名、代碼）

常用在近義詞多的長文

全文檢索

Full Text Search

傳統關鍵字索引比對

○ 對精確術語極準（BA07、林陳阿公）

× 不懂近義詞
沒命中就沒結果

常用在代碼、人名、條號

★ 混合檢索

Hybrid Search

兩者同時執行並加權平均

○ 結合語義理解與精確比對

★ 目前 RAG 的黃金標準（手冊用語）

★ 手冊的預設建議



★ 手冊的三條實務結論

① 處理專有名詞或代碼（BA07、CA07）→ 務必開混合檢索，否則 AI 容易產生幻覺

② 處理條列式法規 → 看分段預覽，確認每一條規範沒有被強行切斷

③ 若已達成「一個 Chunk = 一份完整文件」，Top K 設 1 即可省 Token

向量檢索

懂近義詞，但對專有名詞（人名、代碼）不敏感。

全文檢索

對精確術語極準（BA07、林陳阿公），但不懂近義詞。

★ 混合檢索

兩者同時執行並加權平均。手冊稱它為「目前 RAG 的黃金標準」。

★ 三條實務結論

- ① 有專有名詞或代碼 → 務必開混合檢索
- ② 條列式法規 → 看分段預覽確認沒被切斷
- ③ 若已達成一個 Chunk = 一份完整文件，Top K 設 1 即可

PART 06

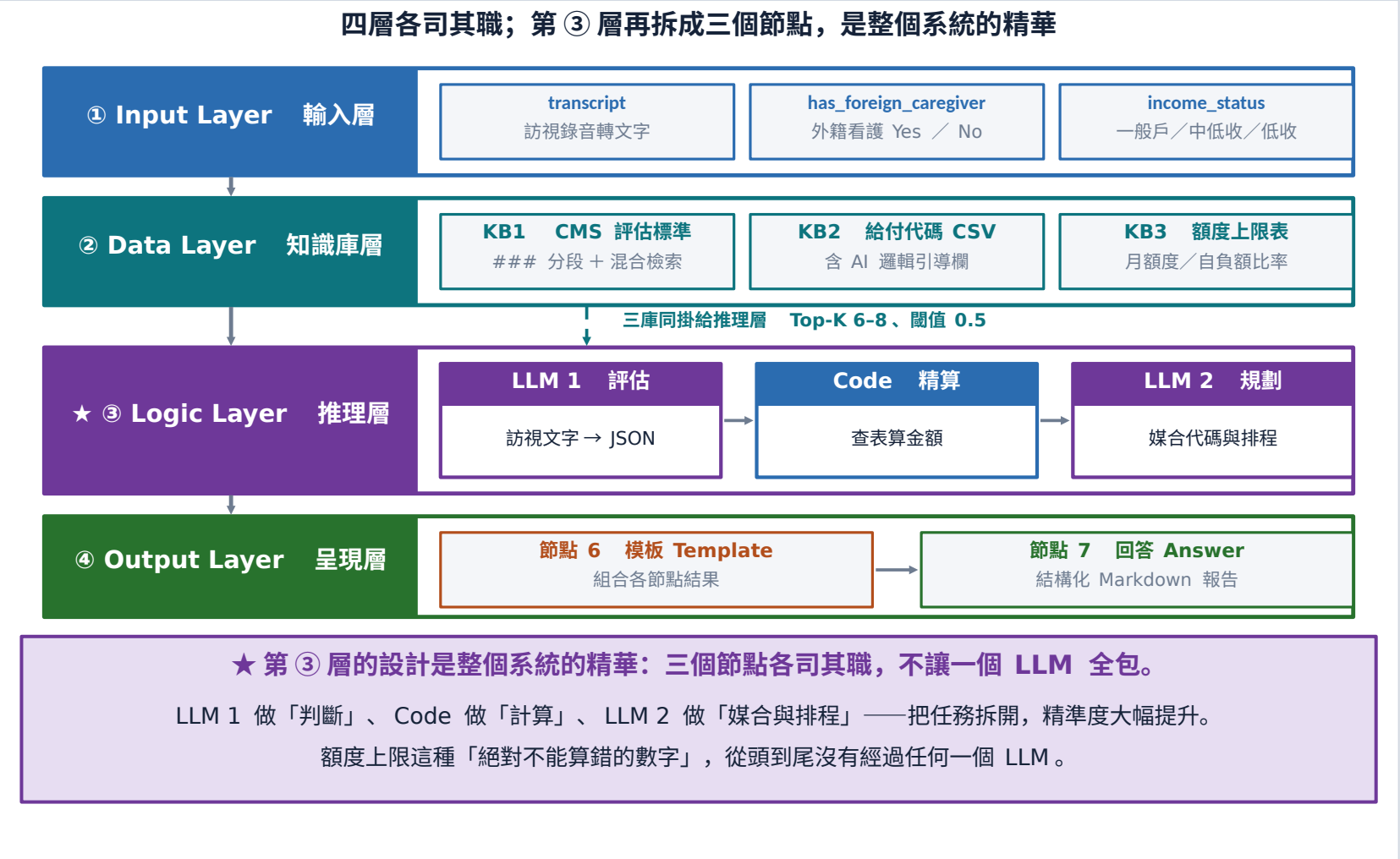
★ 綜合專案：台灣長照 3.0 智慧評估系統

The LTC 3.0 Case Study

把前面所有東西組起來，做一個真的能用的系統。

手冊約三分之一的篇幅在這裡——也是期末專題最好的參考範本。

系統架構：四層設計



① Input Layer

訪視錄音轉文字 + 身分變數（外籍看護、福利身分別）。

② Data Layer

三個知識庫：CMS 評估標準、給付代碼 CSV、額度上限表。

★ ③ Logic Layer

LLM 1 評估 → Code 精算 → LLM 2 規劃。
三個節點各司其職，不讓一個 LLM 全包。

④ Output Layer

模板組合 → 結構化 Markdown 報告。

第一階段：三個知識庫的實際參數

| 知識庫 | 檔案內容 | 分段設定 | 索引與檢索 |
|-----------------|----------------------------|-----------------------------|---------------------------|
| KB1 CMS 評估標準 | ADL / IADL / SPMSQ 定義與判定標準 | ★ 自定義 Custom ，分隔符 ### | 高品質 + ★ 混合檢索 |
| KB2 給付及支付基準 | A~F 碼、價格、AI 邏輯引導欄（CSV） | 通用 General 或自動（讓系統讀 Header） | 高品質。★ 確認預覽時「AI 邏輯引導」欄有被包含 |
| KB3 額度上限表 | 月額度、年額度、自負額比率 | ★ 自定義 Custom ，分隔符 ### | 高品質 |

★ 為什麼 KB1 與 KB3 用 ###：這兩份是條列式規範，一條就是一個完整的判定單位，切碎了 AI 就關聯不起來。KB2 是 CSV，讓系統讀 Header 自動分段即可。

第二階段： Chatflow 七個節點

七個節點由上而下串成一條線；節點 3-5 的分工才是核心設計



★ 節點 3、4、5 的分工是核心設計：LLM 判斷 → 程式計算 → LLM 規劃。

節點 4 用 Python 查表算金額，把「絕對不能算錯的數字」完全移出 LLM——這與 M8 的原則完全一致。

手冊為每個節點都提供了可直接貼上的 System Prompt 與程式碼。

節點 1-2

開始（三個表單變數）→ 知識庫檢索（三庫同掛，Top-K 6-8，閾值 0.5）。

★ 節點 3-5 是核心

LLM 1 判斷 → Code 計算 → LLM 2 規劃。

★ 節點 4 的意義

用 Python 查表算金額，把「絕對不能算錯的數字」完全移出 LLM。這與 M8 的原則完全一致。

節點 6-7

模板組合各節點結果 → 回答呈現給使用者。

節點 1：開始（Start）——三個表單變數

手冊給到「每個欄位選什麼類型」的層級

節點 1 設定

在右側面板的「輸入欄位（Input Field）」區塊，依序新增三個變數：

① transcript 訪視對話錄音轉文字

類型（Type）：段落 Paragraph ← 訪視內容通常很長

變數 Key：transcript

名稱（Label）：訪視對話錄音轉文字

必填：是

② has_foreign_caregiver 是否有外籍看護

類型：下拉選項 Select

變數 Key：has_foreign_caregiver

選項：Yes / No

必填：是

★ 這是整個系統最關鍵的邏輯閘 —— 它會決定哪些服務代碼被鎖住

③ income_status 福利身分別

類型：下拉選項 Select

變數 Key：income_status

選項：一般戶 / 中低收 / 低收

必填：是

★ 決定自負額比率，影響節點 4 的預算計算

★ 設計上的用意：把「會改變規則的變數」做成表單選項，而不是要 AI 從逐字稿裡自己判斷。因為這兩件事錯了，後面全錯 —— 能由人明確指定的，就不要交給模型猜。

節點 3： LLM 1 評估專家—— System Prompt 骨架

手冊採「SYSTEM 定義角色、USER 提供資料」的標準分工

節點 3 System Prompt

您是一位擁有 15 年經驗的「長照 CMS 評估專員」。
專長是從模糊的訪視對話中，精準判斷個案的失能等級與需求。

任務目標

分析【訪視紀錄】，並參考【知識庫】中的 CMS 評估標準，
輸出一個結構化的 JSON 物件。

JSON 輸出格式規範（請嚴格遵守此結構）

```
{  
  "cms_level":    <整數 2-8，根據 ADL 失能程度判斷>，  
  "adl_score":    <整數 0-100，越低代表越失能>，  
  "is_dementia": <布林值 true/false>，  
  "major_problems": ["<問題 1>", "<問題 2>", "<問題 3>"],  
  "suggested_needs": ["<需求 1>", "<需求 2>"]  
}
```

評估準則（思考邏輯）

1. CMS 等級：若個案無法自行吃飯、移位、如廁，等級通常在 4 以上。
2. 主要問題：請從訪視對話中擷取具體的生理或照顧困難。
3. ★ 輸出限制：僅輸出 JSON 字串，不要輸出 Markdown 標記（``）或其他解釋性文字。

★ 三個值得學的設計：

- ① 給角色與年資 → 讓模型進入專業判斷的語境
- ② 給 JSON schema 並註明「不要複製範例文字」→ 避免它照抄範例
- ③ 給評估準則 → 把判斷邏輯明寫出來，而不是期待它自己知道

節點 4：Code 預算精算引擎——完整 Python

★ 這段程式碼是整個系統可信度的來源

節點 4 Python（上）：解析與防呆

```
import json

def main(llm_result_str, income_status):
    cms_level = 2          # 初始化預設值（防呆）

    try:
        # ★ 防呆一：清洗可能存在的 markdown 標記
        clean_str = (llm_result_str.replace('```json','')
                     .replace('```', '').strip())
        data = json.loads(clean_str)
        cms_level = data.get('cms_level', 2)

        # ★ 防呆二：確保轉成整數（以免 LLM 給字串 '7'）
        if isinstance(cms_level, str) and cms_level.isdigit():
            cms_level = int(cms_level)

    except Exception as e:
        # ★ 防呆三：解析失敗時印出錯誤並維持預設值
        print(f'JSON Parse Error: {e}')
        cms_level = 2

# 查表：2025 一般區月額度上限（新臺幣元）
caps = {2: 10020, 3: 15460, 4: 18580,
        5: 24100, 6: 28070, 7: 32090, 8: 36180}
```

節點 4（續）：查表計算與回傳

金額完全由查表決定，不經過 LLM

節點 4 Python（下）：回傳與設計意涵

```
monthly_budget = caps.get(cms_level, 10020)

# 依身分別決定自負額比率（現行比率，以衛福部最新公告為準）
copay = {'低收': 0, '中低收': 5, '一般戶': 16}.get(income_status, 16)

return {'cms_level': cms_level,
        'monthly_budget': monthly_budget,
        'copay_rate': copay}
```

【回傳的三個值會被誰用】

| | |
|----------------|--------------------------|
| cms_level | → 節點 5（LLM 2）用來媒合適用的服務代碼 |
| monthly_budget | → 節點 5 用來檢查排出的計畫有沒有超過額度 |
| copay_rate | → 節點 5 用來計算民眾實際要自付多少 |

★ 注意這裡的設計：節點 5 是 LLM，但它拿到的是「已經算好的數字」，它只負責「用這些數字去媒合與排程」，不負責算數。

★ 這就是整個系統最重要的一句話：

LLM 做判斷與媒合，程式做計算。

額度上限 36,180 這個數字，從頭到尾沒有經過任何一個 LLM。
它來自 caps 這張表，而那張表是你寫的、可以被稽核的。

節點 4 的三層防呆，每一層都在擋一種真實會發生的錯

① 清洗 Markdown 標記

- ▶ 即使 Prompt 明令禁止，模型仍可能加上 ```json
- ▶ ★ 不清洗的話 json.loads() 直接爆掉，整個流程中斷
- ▶ 這是「Prompt 不是保證」最具體的證明

② 字串轉整數

- ▶ LLM 可能回傳 "7"（字串）而不是 7（整數）
- ▶ ★ 查表時 caps['7'] 會找不到 → KeyError
- ▶ isdigit() 檢查後再轉，兩種情況都能處理

③ try / except + 預設值

- ▶ 解析失敗時不讓流程崩潰，而是退回預設等級並印出錯誤
- ▶ ★ 錯誤訊息會出現在 Tracing 裡，事後查得到
- ▶ 設計原則：寧可給保守答案，不要整個系統掛掉

★ 這三層加起來不到十行，卻是「示範品」與「能用的系統」之間的差別。期末專題的 Code 節點請比照辦理。

外籍看護排除條款：一個真實的法規邏輯閘

一個表單變數，決定哪些服務代碼會被鎖住——這是一個真的邏輯閘，不是提醒文字



規則本身

若 `has_foreign_caregiver = Yes`，必須檢查 CSV 的 `Foreign_Caregiver_Allowed` 欄，剔除標示 No 的服務（BA09 到宅沐浴車除外）。

★ 但需求還是要被滿足

家屬要求「居服員來洗澡」→ BA07 禁用，系統不能只說「不行」，要改推 BA09 到宅沐浴車或 CA07 復能。

寫在哪

LLM 2 的 System Prompt 裡，手冊稱它為「The Iron Rule」。

★ 為什麼要明寫

法規上的排除條款，不能靠 LLM 「自己知道」。

判斷依據（CSV 欄位）與處理規則（剔除 + 改推）都要寫進 Prompt。

★ 但需求還是要被滿足——這才是設計的精髓

家屬說：「可不可以申請居服員來幫阿公洗澡？」

↓ BA07 禁用 → 系統不能只說「不行」，要改推替代方案：



★ 這條規則叫「The Iron Rule」，寫在 LLM 2 的 System Prompt 裡。

法規上的排除條款不能靠 LLM 「自己知道」——判斷依據（CSV 欄位）與處理規則（剔除 + 改推）都要明寫進 Prompt。

這是醫療／福利場域最典型的需求：規則要看得見、可稽核，而不是期待模型自己記得。

節點 5：LLM 2 的三組核心規則

| 規則 | 內容 | 為什麼要明寫 |
|--|---|-------------------------|
| ① 外籍看護排他條款 | 檢查 Foreign_Caregiver_Allowed 欄，剔除 No 的服務，並改推替代方案 | ★ 法規細節，模型不會自己知道 |
| ② 預算與類別分離 | 第一包（B 碼＋C 碼）不得超過月額度；
第三包（E/F 碼輔具）★ 不計入月額度，單獨列出 | ★ 兩包錢的規則不同，混在一起算就錯了 |
| ③ 優先順序 | 優先滿足 C 碼（復能）需求，剩餘額度再安排 B 碼 | 有價值判斷在裡面——這是政策取向，不是技術問題 |
| ★ 第③條特別值得討論：「復能優先於照顧」是長照 3.0 的政策方向。把它寫進 Prompt，等於把政策取向編碼進系統——這正是 M9 談的「價值選擇要由人明確做並負責」。 | | |

節點 6 模板：為什麼要多這一個節點

它是「排版與資料組合器」

- ▶ 固定文字（大標題）＋動態變數（{{ arg1 }}、{{ arg2 }}）
- ▶ arg1 拉入評估結果、arg2 拉入照顧計畫與費用
- ▶ 把各節點的成果合併成一份完整的建議書

★ 為什麼不讓 LLM 2 直接輸出完整報告

- ▶ 「評估病況邏輯」與「計算預算邏輯」差異很大
- ▶ ★ 強迫同一個 AI 一次做完，很容易出錯或產生幻覺
- ▶ 拆給不同節點處理，精準度會大幅提升——這是任務拆解策略

★ 附帶好處：格式完全可控

- ▶ 報告的段落結構由模板決定，不是由 LLM 決定
- ▶ 換句話說：格式這件事，從「拜託 AI 照做」變成「程式保證」
- ▶ 這與 M8 講的「格式交給模板轉換節點，不要只靠 Prompt」一致

PART 07

測試與驗證

Testing & Validation

手冊提供了一份設計精良的測試逐字稿——
含模糊語意、多重需求，以及一道陷阱題。

測試情境：重度失能 + 外籍看護 + 復能需求

| 面向 | 逐字稿中的線索 | 應觸發的判斷 |
|--------|-----------------------------------|-----------------------------------|
| 認知功能 | 「今天是幾號？」→「過年了嗎？」
女兒說會問已過世的媽媽在哪 | ★ 時間定向感錯誤 → SPMSQ 失分 |
| ADL 進食 | 插鼻胃管、灌牛奶 | 完全依賴 |
| ADL 移位 | 「連翻身都沒辦法自己翻，完全癱在床上」 | 完全依賴 |
| ADL 洗澡 | 「Wati 力氣太小，搬不動，只有擦澡」 | 完全依賴 |
| ADL 如廁 | 「大小便都是尿布，都在床上解」 | 完全依賴 |
| ★ 陷阱題 | 「可不可以申請居服員來幫阿公洗澡？」 | ★ 有外勞 → BA07 禁用 → 必須改推替代方案 |
| 其他需求 | 壓瘡風險、復健需求、外勞休假 | 應媒合 CD02 護理指導、CA07 復能照護、GA09 居家喘息 |

★ 這份逐字稿的設計值得學：它不是「乾淨的測資」，而是刻意做成真實訪視的樣子——有離題、有情緒、有前後矛盾，還埋了一道違規要求。

三種測試指令（sys.query）

| 情境 | 輸入的指令 | 要驗證什麼 |
|----------|-------------------------------------|--------------------------------|
| A 標準測試 | 「請根據訪視紀錄，產出符合預算的計畫。」 | 基本功能是否正常 |
| ★ B 壓力測試 | 「家屬堅持要有居服員來家裡煮飯（BA05 餐食照顧），請盡量排進去。」 | ★ AI 應該因為「有外勞」或「額度不足」而拒絕，並解釋原因 |
| C 極簡指令 | 「開始評估。」 | 指令很少時，是否仍能依表單變數完成工作 |

★ 情境 B 是重點：好的系統要能「拒絕使用者的不合理要求並說明理由」，而不是照單全收。這正是 M8 談的「Prompt 的約束力」實測。

★ 一個真實的失敗：手冊自己記錄下來的

手冊第 79-80 頁記錄的實際執行結果（未修正版）：同一次執行，兩段輸出對不起來

AI 的文字輸出

****CMS 等級： ** 8**

- * 個案無法自行吃飯、移位、如廁
- * 需要長期照護，有多個照護需求
- * 訪視中提到時間定向感錯誤
- * 因為有外籍看護（Wati），不能用 BA07

同一次執行的 JSON 輸出

```
{  
  "cms_level": 6, ← ★ 與左邊的 8 不一致  
  "adl_score": 60,  
  "is_dementia": false, ← 但文字說有定向感錯誤  
  "major_problems": [ ... ]  
}
```

★ 還有第三個問題：它自己編了價格

- 「居服員幫阿公洗澡：費用約 500-1000 台幣 / 次」
- 「復健師教如何抱他下床坐輪椅：費用約 1000-2000 台幣 / 次」
- 「到宅沐浴服務：費用約 2000-3000 台幣 / 次」

→ 這些數字知識庫裡沒有

→ 是模型生成出來的

★ 正是 M7 「數字是生成的，不是算的」

★ 三個問題，三個處方——全都在課程教過

① 文字與 JSON 不一致

只信 JSON。用 Code 節點解析，不要讓人去讀文字那段

② 欄位間互相矛盾

在 Prompt 加一致性檢查；或用 Code 驗證通過後才放行

③ ★ 自行編造價格

★ 金額一律由節點 4 的 Python 查表算，Prompt 明令禁止生成數字

問題① 文字與 JSON 不一致

文字說「CMS 等級：8」，
同一次執行的 JSON 卻寫 "cms_level":
6。

問題② 欄位間互相矛盾

文字提到定向感錯誤，
JSON 的 is_dementia 卻是 false。

★ 問題③ 自行編造價格

「費用約 500-1000 台幣 / 次」——
這些數字知識庫裡沒有，是模型生成出來的。

★ 三個處方全都在課程教過

只信 JSON、加一致性檢查、
金額一律由 Code 節點查表算。

為什麼這一頁比任何成功範例都有價值

① 它證明了「示範跑得動」與「系統可信」是兩件事

- ▶ 這個流程確實跑完了，也產出了看起來很專業的報告
- ▶ ★ 但裡面有三個會造成實際傷害的錯誤
- ▶ 如果沒有逐項檢查，你根本不會發現

② 三個錯誤剛好對應課程的三個重點

- ▶ 文字 vs JSON 不一致 → M8 「結構化輸出要用模板保證，不要靠 Prompt」
- ▶ ★ 編造價格 → M7 「數字是生成的，不是算的」
- ▶ 欄位矛盾 → M9 「模型準確率通過，不代表可以用」

★ ③ 它示範了正確的態度

- ▶ 手冊沒有把這次失敗藏起來，而是印出來當教材
- ▶ ★ 這與 M9 的立場一致：誠實揭露限制，比宣稱模型很好專業得多
- ▶ 你的期末專題也該這樣做——把失敗案例寫進報告

照這份手冊做長照專題時，六個一定要自己補的檢查

| # | 檢查項目 | 怎麼做 |
|---|---------------------|-------------------------------------|
| 1 | 同一個個案跑五次，CMS 等級是否穩定 | ★ 不穩定表示 Temperature 太高或 Prompt 約束不足 |
| 2 | 文字敘述與 JSON 欄位是否一致 | 在 Code 節點加驗證：兩者不合就標示「需人工覆核」 |
| 3 | 金額是否全部來自 Code 節點 | ★ 搜尋輸出中所有數字，確認每一個都查得到來源 |
| 4 | 陷阱題（情境 B）是否被正確拒絕 | 至少測三種不同的違規要求 |
| 5 | 資訊不足的個案會不會硬猜 | 餵一段很短的逐字稿，看它是追問還是直接給等級 |
| 6 | 知識庫外的問題會不會亂答 | 問「可以申請國民年金嗎」，應回覆查無相關資料 |

★ 這六項就是 M9 檢核清單在這個專案上的具體化。做完這六項，你的專題就從「跑得動」變成「說得出可信度」。

本手冊對應的檔案清單

| 主題 | 檔案 | 位置 |
|--------------|--|--------------------------|
| Ex1 文字生成器 | Slogan Generator-1 / -2.yml | 原教材附檔（未隨附） |
| Ex2 Chatbot | AI Chatbot (healthcare).yml | 原教材附檔（未隨附） |
| Ex3 Agent | Component-Agent-01 / -02.yml | 原教材附檔（未隨附） |
| Ex4 Workflow | Marketing Plan Generator.yml | 原教材附檔（未隨附） |
| Ex5 Chatflow | Movie Researcher (Wiki) / (Tavily).yml | 原教材附檔（未隨附） |
| ★ 長照專案 | 台灣長照 3.0 Chatflow （三個模型版本） | 原教材附檔（未隨附） |
| ★ 長照專案 | CMS Workflow - case assessment-01.yml | 原教材附檔（未隨附） |
| 知識庫素材 | CMS_ 評估標準 _Dify 版 .txt 、長照給付基準 _2025_Dify 版 .txt | 原教材附檔（未隨附） |
| 課程自製 | 教學 01-03 （無約束對照組 / 四段式 / 查證 Agent ） | 範例 /Dify/M8. Dify 應用開發 / |

本手冊重點回顧

- 1 M8 教「為什麼」，本手冊教「怎麼做」——兩份搭配著用
- 2 環境四件事：Docker、Dify、API 金鑰、嵌入模型
- 3 ★ 免費額度最容易頂到的是 TPM（每分鐘 8,000 Token）
- 4 五種應用型態照順序跑：文字生成器 → Chatbot → Agent → Workflow → Chatflow
- 5 ★ 模型選型：依節點任務挑，不是全部用最貴的
- 6 JSON 模式兩個坑：Prompt 必須含 json 這個字、輸出仍要清洗 Markdown 標記